

2023/1/4 IBP 112006265 蔡定宇

1.Verilog code

```
module counter (  
    input clk,  
    input reset,  
    input input_signal,  
    output reg [3:0] output_value  
);  
  
reg prev_bit;  
reg is_counting;  
  
always @(posedge clk or negedge reset) begin  
    if (~reset) begin  
        output_value <= 0;  
        is_counting <= 0;  
        prev_bit <= 0;  
    end else begin  
        if (!is_counting && prev_bit && input_signal) begin  
            is_counting <= 1;  
            output_value <= (output_value == 9) ? 0 : output_value + 1;  
        end else if (is_counting && output_value == 9) begin  
            is_counting <= 0;  
            output_value <= 0;  
        end else if (is_counting) begin  
            output_value <= output_value + 1;  
        end  
        if (!is_counting) prev_bit <= input_signal;  
    end  
end  
  
endmodule
```

2. Source code of Verilog testbench file

```
`timescale 1ns / 1ps

module counter_tb;

    reg clk;
    reg reset;
    reg input_signal;
    wire [3:0] output_value;
    integer index;
    reg [29:0] test_input_string = 30'b000100011000000000001110000000;
    reg [29:0] test_reset_string = 30'b101111111111111111111111011111;

    counter uut (
        .clk(clk),
        .reset(reset),
        .input_signal(input_signal),
        .output_value(output_value)
    );

    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end

    initial begin
        clk = 0;
        reset = 0;
        input_signal = 0;
        reset = 1;

        for (index = 29; index >= 0; index = index - 1) begin
            input_signal = test_input_string[index];
            reset = test_reset_string[index];
            #20;
        end
    end
endmodule
```

```
end

#100;
$finish;
end

initial begin
    $dumpfile("counter_tb.vcd");
    $dumpvars(0, counter_tb);
end

endmodule
```

3. Simulation results

